

oauth2-passkey

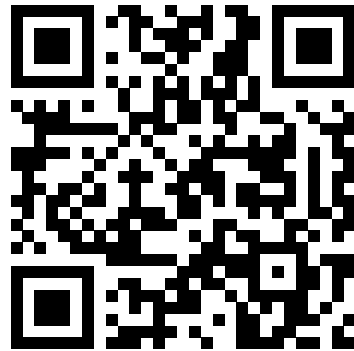
Passwordless Authentication Library for Rust
Kimitoshi Takahashi

Tokyo Rust Show & Tell | 2026/03/31

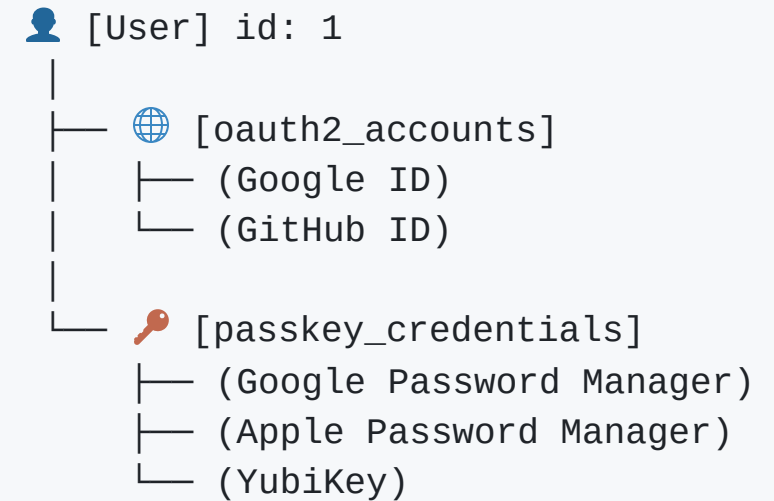
Live Demo

Demo

1. Google OAuth2/OIDC Login to create a user
2. Passkey Promotion -> register
3. Login with either Passkey or Google OAuth2



Try it out & give me feedback!



Motivation

I wanted to build **exactly what you just saw** — in Rust.

- **OAuth2/OIDC**: delegate auth to trusted providers — no passwords to manage
- **Passkey**: phishing-resistant, biometrics/hardware key — no server-side secrets
- No integrated Rust/Axum library existed → built one → published to **crates.io**

Agenda

1. Using the Library
2. Multi-DB Storage Support
3. Wrap-up

Using the Library

.env Setup (Minimal)

```
ORIGIN='http://localhost:3001'

# Google OAuth2 credentials
OAUTH2_GOOGLE_CLIENT_ID='xxx.apps.googleusercontent.com'
OAUTH2_GOOGLE_CLIENT_SECRET='xxx'

# SQLite + in-memory cache (no DB setup required)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'
GENERIC_CACHE_STORE_TYPE=memory
GENERIC_CACHE_STORE_URL='memory'
```

- Data store: SQLite, PostgreSQL, MySQL
- Cache store: memory, Redis

How to Use

```
use oauth2_passkey_axum::{
    AuthUser, oauth2_passkey_full_router, // 1, Import
};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?;          // 2. Initialize

    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 3. Merge router

    // Auth endpoints are now at /o2p/*
    spawn_http_server(3001, app).await?;
    Ok(())
}
```

Under `/o2p/*`, we have:

- OAuth2 endpoints and Passkey endpoints
- Built-in login UI, account management, and admin panel

Page Protection: AuthUser Extractor

```
// Required auth - redirects to login if not authenticated
async fn protected(user: AuthUser) -> impl IntoResponse {
    format!("Hello, {}!", user.label)
}

// Optional auth - allows anonymous access
async fn public(user: Option<AuthUser>) -> impl IntoResponse {
    match user {
        Some(u) => format!("Hello, {}!", u.label),
        None    => "Hello, anonymous!".to_string(),
    }
}
```

Implemented via Axum's `FromRequestParts` trait:

- `AuthUser` → GET: redirect to login, others: 401
- `Option<AuthUser>` → `OptionalFromRequestParts` impl returns `None` on failure — no redirect

Limitation: always hits DB, GET always redirects. Middleware gives more control: skip DB, or return 401 even on GET.

Page Protection: Middleware

Middleware gives more control: **skip DB, or return 401 even on GET.**

```
let app = Router::new()
  .route("/api/1", get(h1)
    .route_layer(from_fn(is_authenticated_401))) // <-- No DB query
  .route("/api/2", get(h2)
    .route_layer(from_fn(is_authenticated_user_401))); // <-- DB query, user info available
```

```
// In handler: access user or CSRF token via Extension
async fn h1(Extension(csrf): Extension<CsrfToken>) { ... }
async fn h2(Extension(user): Extension<AuthUser>) { ... }
```

Storage & LazyLock Pattern

Switch DB by Changing .env

```
# SQLite (dev/demo - no setup required)
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'

# PostgreSQL
GENERIC_DATA_STORE_TYPE=postgres
GENERIC_DATA_STORE_URL='postgres://user:pass@localhost:5432/mydb'

# MySQL / MariaDB
GENERIC_DATA_STORE_TYPE=mysql
GENERIC_DATA_STORE_URL='mysql://user:pass@localhost:3306/mydb'

# Cache: in-memory or Redis
GENERIC_CACHE_STORE_TYPE=memory      # or: redis
GENERIC_CACHE_STORE_URL='memory'     # or: redis://localhost:6379
```

No code changes. Just swap the env vars and restart.

Wrap-up

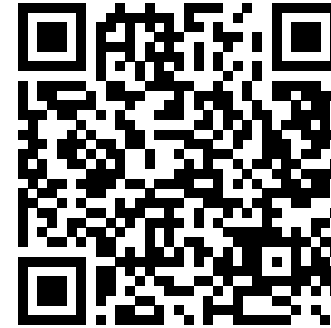
Summary

- **Easy:** Add passwordless auth to your Axum app in minutes — Built-in UI included
- **Secure:** Passkey (phishing-resistant) + OAuth2, with CSRF protection and secure session cookies
- **Flexible:** Switch between SQLite, PostgreSQL, MySQL, and Redis with a single `.env` change

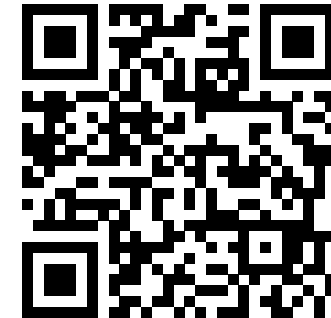
Thank You! / Questions?

About me:

- **Kimitoshi Takahashi**
- Self-employed, reskilling in Rust (3rd year)
- Let's start a startup together!



GitHub



Contact